FABLE: Frontier-Driven Orchestration for Distributed Complex Event Recognition in IoT Systems

***Abstract*—Complex-event detection requires observations distributed across sensors, locations, and time. As the complex event progresses, new observations may change which sensor is relevant, which past or future interval matters, or what event interpretations are more likely. We call these *dynamic evidence requirements*. Addressing dynamic evidence requirements requires adapting sensing and computation based on current observations and operational conditions. Existing complex-event and edge-analytics systems often adapt model configurations or placement within configured queries or pipelines, rather than revising the detection plan as the complex event evolves.**

We present *FABLE*, a runtime that adapts the detection plans for each ongoing complex event. *FABLE* represents complex events as temporal AND/OR graphs and uses event progress to determine which observations can advance recognition. A bounded multi-label search selects a plan for executing detection over sensing sources, inference pipelines, data representations, and physical devices throughout the complex event detection process. Each new observation enables targeted activation and placement of detection functions, retrospective inference over retained data, and changes in which complex event alternatives are pursued.

We implement *FABLE* in Python using containerized inference modules and coordinated node agents. We evaluate *FABLE* with recorded multimodal traces under controlled network and workload changes against always-on, fixed-pipeline, and resource-adaptive baselines, measuring recognition quality, timeliness, computation, and communication.

***Index Terms*—complex event processing, distributed sensing, edge computing, multimodal sensing, model orchestration, event-time processing**

I. Introduction

Timely detection of complex events can guide operational decisions by human operators and autonomous systems in tactical missions, urban public safety, and transportation management—for example, where to direct personnel, which locations to monitor, or how a mobile platform should respond. Complex event detection in these deployments relies on sensory observations derived from cameras, microphones, and other sensors distributed across locations and time. As a convoy moves through a road network, the route revealed by incoming observations may determine which camera should be activated next to continue convoy tracking. At a bank, a detected gunshot may make earlier recordings relevant for identifying an arriving vehicle and require later monitoring for the identified vehicle's departure. A package exchange may occur either as a direct handoff or as a drop-off followed by a later pickup. The convoy, robbery, and package-exchange examples show that incoming observations can do more than advance recognition of a complex event. Incoming observations can also change what additional observations and computations are needed for the complex event. A complex event detection system may therefore need to revise where observations are collected and processed, when sensing and inference are activated, which past or future intervals are examined, and which interpretation of the complex-event request remains viable. We call the changing requirements for obtaining and interpreting observations *dynamic evidence requirements*. Figure 1 summarizes the three recurring forms of dynamic evidence requirements: spatial progression, temporal evidence dependence, and alternative event manifestations. These requirements demonstrate that how complex event progress should be treated as a control signal for sensing and computation.

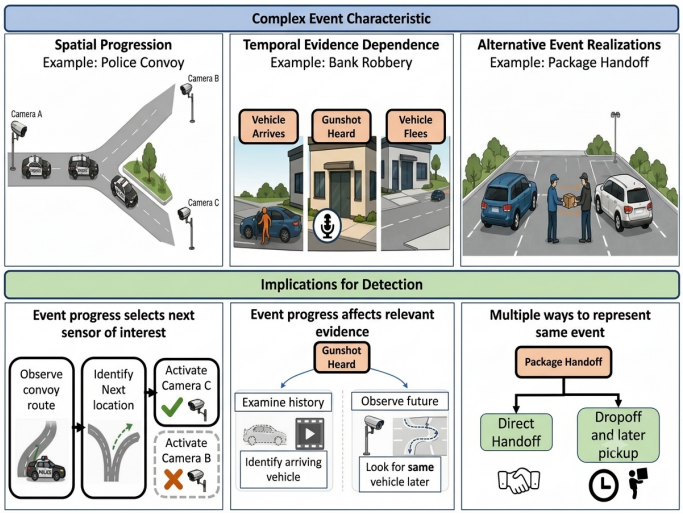

Fig. 1. Dynamic evidence requirements in complex-event detection. Spatial progression changes the next relevant sensing location, temporal evidence dependence changes which past or future observations matter, and alternative event manifestations require multiple valid recognition paths.

The distinguishing challenge is not that complex events may span one sensor, multiple sensors, or long time intervals. Classical complex-event processing systems support temporal and relational patterns over event streams [1], [2], [3], while spatiotemporal monitoring and video or multimodal recognition systems extend such patterns across sensors and locations [4], [5], [6], [7], [8]. These systems establish that rich spatiotemporal complex-event structures can be represented and detected.

The remaining challenge is that the detection plan may need to change as the complex event evolves. Adaptive complex-

event and video-analytics systems can reconfigure operators, model choices, and placements as runtime conditions change [9], [10], [11], [12], [8], [13], but primarily adapt the execution of a configured query or analytics graph. Dynamic evidence requirements additionally require complex-event progress to revise which sensors, time intervals, and event alternatives should be pursued. This distinction is especially important in tactical and urban deployments, where mobile entities cross partial sensor coverage, retained observations may expire, and network, compute, and sensor availability vary. Complex event progress determines which observations and computations are needed, whereas operational conditions determine when and where that work can execute.

We present *FABLE* , a runtime that uses complex-event progress to orchestrate distributed sensing and computation. *FABLE* represents a request as a shared temporal AND/OR event graph and maintains the progress of each ongoing complex event. From that progress, *FABLE* identifies the detection conditions that can currently advance recognition, which we call the *active frontier*. Rather than fixing and placing an entire sensing pipeline in advance, *FABLE* plans only through the next semantic checkpoint, where a new observation may identify an entity, resolve an alternative event, or change the observations needed next. The runtime then updates the corresponding complex event progress and plans again.

The progress-driven loop directly addresses the requirements in Fig. 1. For spatial progression, *FABLE* prioritizes sensing and inference at likely next locations when route or deployment knowledge is available. For temporal evidence dependence, *FABLE* issues bounded requests over retained observations or defined future intervals. For alternative event manifestations, *FABLE* continues the complex-event paths that remain viable and avoids work for paths that have been ruled out.

For the currently relevant observation requirements, the *FABLE* planner uses bounded multi-label search to construct a feasible plan over sensing sources, inference pipelines, data representations, and execution placements through the next semantic checkpoint. The planner evaluates alternatives under current deadlines and resource conditions. Node agents execute the selected plan, and newly produced observations update the corresponding complex-event match and trigger replanning.

This paper makes three contributions:

- We design a progress-aware runtime representation that combines a shared temporal AND/OR event graph with lightweight state for an ongoing complex event. The active frontier identifies the conditions that can currently advance recognition, while semantic checkpoints bound how far the system plans before updating the observation requirements.
- We design checkpoint-bounded planning for dynamic evidence requirements. A bounded multi-label search constructs feasible plans across sensing sources, inference pipelines, data representations, and execution placements under deadlines and resource constraints. Replanning at semantic checkpoints supports likely-next-location activation, bounded retrospective and prospective processing, and alternative complex-event manifestations.
- We implement *FABLE* in Python using containerized providers and node agents coordinated through MQTT, and evaluate the runtime using trace-driven multimodal complex-event scenarios, heterogeneous execution profiles, and controlled network and workload changes. We compare against always-on, static, whole-event, and task-level adaptive execution to measure whether progress-aware orchestration preserves recognition quality and timeliness while reducing computation and communication. We show **[QUALITY RESULT]** while reducing **[RESOURCE RESULT]**.

By making the observation plan contingent on complex event progress, *FABLE* supports complex-event detection in sensing deployments where sensing and computation must adapt to changing event semantics and operational conditions.

II. Problem Setting

a) Complex-event scope: FABLE consumes typed, timestamped observations derived from video, audio, and other sensor streams. A complex-event request may relate repeated observations from one sensing source over a long interval, observations from several locations over a short interval, or observations distributed across both locations and extended time. We discuss these different types of complex events in J. However, these cases describe the *spatiotemporal footprint* of a request; they do not by themselves require a dynamic observation plan. Any of them can be recognized by a system that continuously processes a fixed set of streams. Our focus is the additional case in which observations associated with an ongoing complex-event match change which sensing locations, time intervals, computations, or authored event alternatives should be pursued next. We refer to these changing needs as *dynamic evidence requirements*.

b) Deployment assumptions: We consider deployments with heterogeneous sensing sources connected to logical device, edge, and cloud compute nodes. A sensing source has a modality and physical coverage region, while a compute node has finite processing, memory, and accelerator capacity. The deployment may also provide qualitative knowledge such as overlapping coverage, adjacency, route direction, or likely downstream locations. Network bandwidth and latency, provider availability, and node load can change during recognition. Sensor data and derived artifacts may be retained only for bounded intervals and may remain local to the node that produced them. Consequently, the location of relevant observations and the location at which inference can run are separate decisions.

c) Recognition state: A complex-event request is compiled into a shared, typed temporal AND/OR graph. The graph expresses semantic predicates, alternatives,

conjunctions, ordering, and temporal windows without committing to particular sensors or models. Each ongoing candidate match is represented by a lightweight *hypothesis* that records the associated entities, satisfied graph nodes, viable alternatives, and event-time state. The *active frontier* of a hypothesis contains the unresolved predicates whose outcomes can currently advance, invalidate, fork, or disambiguate that hypothesis. Different hypotheses over the same request may therefore expose different frontiers. A *semantic checkpoint* is the next boundary whose resolution may change the relevant bindings, event alternative, or future observation requirements.

d) Planning problem: Given the active hypotheses and current deployment conditions, *FABLE* must determine the next observation and computation plan. The plan selects which live or retained sources to examine, which event-time intervals matter, which inference providers and data representations to use, and where the computation should execute. A feasible plan must satisfy provider compatibility, data locality and retention, network reachability, resource capacity, and the latest time at which each result can still affect recognition. Rather than fixing the entire sensing pipeline when the request is submitted, *FABLE* plans only through the next semantic checkpoint, incorporates returned observations, and then plans again. The objective is timely, semantically valid complex-event recognition while avoiding sensing, computation, and communication that can no longer contribute to a viable match.

III. System Design

A. Architecture and Control Loop

Figure 2 shows the recurrent *FABLE* workflow. A request enters the *semantic engine*, which resolves an authored complex-event family and constructs a shared temporal event graph. The *Active Frontier Manager* tracks candidate matches over that graph and identifies the semantic predicates that can currently affect each match. Those predicates become provider-independent *predicate demands*. The *execution planner* combines the demands with the provider catalog and current runtime conditions to choose an execution plan. Node agents execute the selected providers on the logical device, edge, and cloud tiers and return typed predicate results. The results update the corresponding match; if the complex event is incomplete, the loop repeats with a new frontier and plan.

The controller is logically centralized for complex-event semantics: it owns request and graph versions, hypothesis state, role bindings, frontier updates, and final complex-event acceptance. Observation production remains distributed. Node agents own sensor access, provider processes, local retained data, and derived artifacts. Raw media therefore need not be continuously centralized; a plan may instead move detections, tracks, identity descriptors, selected media segments, or symbolic predicate results when the selected provider chain permits.

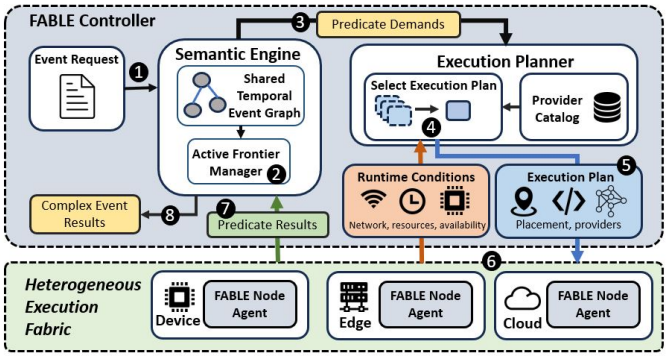

Fig. 2. *FABLE* architecture and recurrent control loop. The semantic engine converts request progress into predicate demands, the execution planner selects provider and placement alternatives under current runtime conditions, and node agents return typed results that update recognition.

Requests are resolved against registered event families and validated parameters. An authored family may contain several semantic branches, such as direct handoff and drop-off/pickup alternatives in a package-exchange request. Request initialization selects the family and its parameters but does not select a runtime branch. Branch viability is determined separately for each candidate match as observations arrive. Details of request schemas and optional language-model assistance are provided in Appendix A.

B. Shared Graph and Occurrence-Specific Frontiers

The shared temporal event graph describes what relationships define the requested complex event without selecting a detector, model, data representation, execution node, or network path. Primitive leaves represent semantic predicates; AND and threshold nodes combine required conditions; OR nodes represent authored event alternatives; and temporal guards express ordering, duration, absence, repetition, and bounded gaps. Sharing the request level graph avoids cloning the full definition for every candidate match. Instead, each hypothesis stores only its associated entities, satisfied nodes, viable branches, temporal state, and observation provenance.

Figure 3 illustrates this distinction for a package exchange. Both hypotheses have observed a person arriving, but only Hypothesis 2 has established that the person holds a package. Hypothesis 1 must therefore continue testing the package condition, whereas Hypothesis 2 can test the direct-handoff and package-drop-off alternatives. A later departure must occur within the authored temporal window after either exchange alternative. The graph is shared, but the active frontier is specific to the observations and bindings of each hypothesis.

The frontier is also the point at which dynamic evidence requirements enter the runtime. Each enabled frontier predicate is compiled into a predicate demand that identifies the relevant hypothesis and entities, the past, present, or future interval to examine, a latest useful completion time, and eligible or preferred sensing

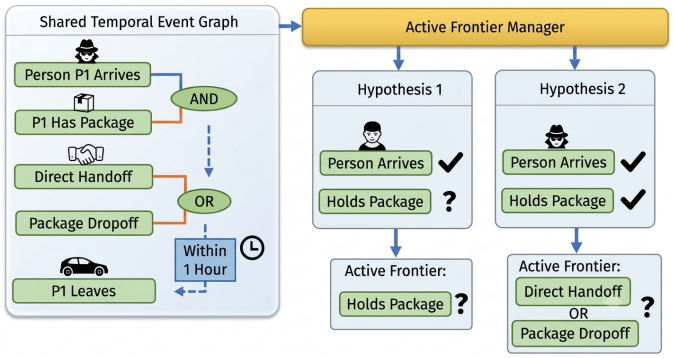

Fig. 3. Shared temporal event graph and occurrence-specific frontier monitoring. Hypotheses maintain different bindings and satisfied conditions over the same graph, so the Active Frontier Manager identifies different predicates that can advance each candidate match.

sources. For spatial progression, deployment or route knowledge can rank likely next sensing locations. For temporal evidence dependence, a later trigger can create both a bounded retrospective demand over retained observations and a prospective demand for later monitoring. When a provider must preserve state across checkpoints, the demand can request a typed artifact such as a track, trajectory, identity descriptor, selected crop, or audio feature window.

C. Checkpoint-Bounded Planning and Execution

The execution planner translates the current predicate demands into physical choices. The provider catalog describes which semantic predicates each provider or provider chain can produce, the required input and output types, eligible placements, resource and latency profiles, replayability, and artifact compatibility. The planner combines that information with live and retained sources, node capacity, network conditions, and available continuation artifacts. A physical alternative therefore jointly specifies an observation source, inference pipeline, data representation, execution placement, required transfers, and continuation outputs. These dimensions cannot always be selected independently: same-camera following may use local tracks, whereas cross-camera association may require an identity descriptor at another node.

FABLE uses bounded multi-label search to construct a feasible plan through the next semantic checkpoint. The search extends partial plans one demand at a time, removes alternatives that violate hard compatibility, locality, retention, capacity, quality, or deadline constraints, prunes dominated partial plans, and retains a bounded set of candidates. Planning only to the next checkpoint limits planning cost and avoids committing to work whose relevance may change after the next observation. The selected execution plan contains the provider steps, placements, inputs, transfers, and artifacts needed for that checkpoint; detailed search and ranking rules appear in Appendix D.

Node agents validate and execute the selected steps, either launching a containerized provider or attaching to an already running compatible service. Returned predicate results include request, graph, hypothesis, binding, and event-time attribution. The semantic engine accepts only results that still match the current hypothesis version and observation interval. Accepted results update the hypothesis and trigger replanning. A provider process or retained artifact may be reused only when source, interval, configuration, representation, and output compatibility agree; work is detached when the associated branch becomes impossible, the observation window expires, or the result can no longer arrive in time. Resource or network changes can invalidate the current physical plan without changing the semantic graph, allowing the planner to select a different provider, placement, or retained-data path.

IV. Implementation

a) Semantic engine: We implement *FABLE* in Python. A complex-event request is compiled into a shared temporal AND/OR graph, while the semantic engine maintains lightweight state for each ongoing match. As predicate results arrive, the engine updates the entities associated with the match, records which graph conditions have been satisfied, and derives the active frontier and next semantic checkpoint. Version numbers and stable result identifiers prevent stale or duplicate results from changing the current match state.

b) Execution planning: The provider catalog and deployment description are loaded from YAML configuration files. The catalog records which sensing and inference components can satisfy each predicate, the observations required and produced by each component, supported device, edge, and cloud placements, and estimated resource requirements. For each active frontier condition, the execution planner constructs alternatives over sensing sources, inference pipelines, data representations, and execution nodes. A bounded multi-label search removes infeasible and dominated alternatives and returns plans through the next semantic checkpoint. The planner rechecks resource and provider availability before dispatch and can reuse compatible providers that are already running. Additional search parameters and the exhaustive offline oracle are described in Appendix E.

c) Distributed execution: The controller sends execution plans to node agents over MQTT. A node agent starts an approved Docker provider or connects to an existing replay or analytics service on the corresponding node. Providers return predicate results and references to retained intermediate outputs, such as tracks, identity descriptors, or selected sensor segments. Providers do not determine whether a complex event has been recognized; only the semantic engine updates and completes complex-event matches. MongoDB stores controller state, while durable local queues and retained-data indexes support message delivery and retrospective processing. Heartbeats and version checks allow the planner to respond to unavailable nodes, expired observations, and provider failures.

d) Sensing and inference providers.: The prototype includes providers for object detection and tracking, geometric and route relations, retrospective replay, cross-camera association, audio-event detection, audio–visual association, and package-custody reasoning. The specific list of providers is described in IV. Providers exchange common observation and intermediate-output formats, allowing the execution planner to choose among alternative implementations of the same semantic predicate.

e) Evaluation deployment.: Docker Compose assembles the controller, MQTT broker, database, node agents, replay services, and inference providers. Mininet, Open vSwitch, traffic control, and cgroup limits emulate the network and resource characteristics of device, edge, and cloud nodes. The evaluated logical tiers run on one x86 host; therefore, placement experiments measure the imposed heterogeneous resource and network profiles rather than communication among physically separate machines.

V. Related Work

a) Complex-event processing and multimodal recognition.: Classical CEP systems such as SASE and Cayuga detect temporal patterns over structured streams, while later work improves partial-match processing and extends event specifications with spatial relations and distributed monitoring [1], [2], [3], [4], [5]. Video and multimodal systems similarly combine learned perception with symbolic event logic or graph-based queries [6], [14], [15], [7], [16], [17], [18], [19], [20], [21]. These systems provide representations and runtimes for event structure and partial progress, but typically assume that primitive observations are already available or that each request executes as a fixed recognition job. *FABLE* instead uses the grounded progress of each possible occurrence to determine which distributed evidence-producing work is useful next.

b) Cost-sensitive activity inference.: Structured activity-recognition systems have used semantic state to reduce unnecessary visual processing. Amer et al. selectively schedule inference over activity AND/OR graphs [22], [23], including detector and tracker placement within a video volume, while AdaFrame learns which frames to inspect for efficient video recognition [24]. *FABLE* applies the same broad principle at the distributed CE level: an active predicate may be realized through different sensors, modalities, providers, representations, placements, and data paths rather than only through a choice of visual operation or frame.

c) Semantics-driven sensor selection.: Sensor-selection and cross-camera systems exploit learned spatial and temporal correlations to avoid querying every sensor. Vaisenberg et al. schedule sensor collection under resource constraints, [25] and Spatula prunes unlikely cameras and intervals during cross-camera search [26]. Their objective is generally a fixed sensing or tracking task. In *FABLE*, sensor relevance is conditioned on a particular CE occurrence, its bound entities, active predicate, route, and event-time window, and is considered jointly with provider and representation choices.

d) Adaptive CEP, dataflow, and scheduling.: Distributed CEP systems adapt the execution of known event queries as mobility, network conditions, and QoS requirements change. MigCEP migrates stateful operators [9], while TCEP switches among operator-placement mechanisms [10]. Related dataflow and scheduling systems manage event-time progress, state, operator ordering, DAG placement, migration, and shared computation [27], [28], [29], [30], [31], [32], [33], [34]. These systems optimize an already instantiated query or task graph. *FABLE* additionally derives the tasks themselves from entity-bound CE progress and carries semantically typed continuation artifacts between checkpoints.

e) Distributed video analytics and model orchestration.: Video-analytics systems reduce cost through model selection, filtering, approximation, cross-camera coordination, and distributed scheduling [35], [11], [12], [36], [37], [38], [39], [8], [13]. Model-serving and edge runtimes further optimize multi-tenant inference, resource selection, and shared DNN computation [40], [41], [42], [43]. These systems adapt how known analytics execute. *FABLE* also decides whether a predicate demand should exist, which entities and event-time interval it concerns, when its result ceases to be useful, and which artifact form preserves later event processing.

f) Provider lifecycle and construction.: Serverless systems reduce function startup and state-access overhead [44], [45], while VOCAL-UDF uses language models and active learning to construct functions for compositional video queries [46]. *FABLE* instead selects among pre-registered, typed providers at runtime; provider startup, sharing, and retirement are driven by CE progress and evidence validity, while language-model assistance is confined to bounded compilation or slow-path tasks.

FABLE combines established ideas from CEP, selective inference, sensor selection, adaptive placement, and video analytics. Its distinction is that grounded CE progress determines which distributed evidence-producing computation should exist next and how it should be realized through the next semantic checkpoint.

VI. Evaluation

We evaluate three claims. **RQ1** asks whether adapting the observation and execution plans to the progress of each complex-event match preserves recognition quality and timeliness while reducing computation and communication. **RQ2** asks whether checkpoint-bounded, joint planning improves feasibility and cost relative to static and greedy alternatives. **RQ3** asks whether replanning handles changing operating conditions and the spatial, temporal, and alternative observation needs illustrated in Fig. 1.

A. Methodology

a) Corpus and scope.: The evaluation replays camera and audio traces collected in three campaigns. After quality filtering, the corpus contains 83 recommended experiments, 2:23:36 of labeled complex-event intervals, and nine event families spanning single- and multi-location, sequential, concurrent, multimodal, and long-running patterns. Appendix H reports the complete event, campaign, sensor, and duration breakdowns in Tables VI and VII. All recognition and planning experiments are derived from the recorded campaign traces. Planner-only experiments replay frontier states, provider profiles, and deployment conditions from those same workloads.

Topology-based spatial experiments use the 2024 and 2025 fixed-sensor deployments, whose sensor locations are known. The 2026 traces remain eligible for recognition, retrospective processing, and operating-condition experiments but are excluded from topology-based metrics. Mobile sensors are also excluded from spatial denominators until mobile replay is supported.

b) Experiment modes.: End-to-end replay supplies raw streams to the containerized providers and measures provider activation, resource use, communication, and complex-event delay. A common-observation control supplies identical timestamped detections, tracks, associations, audio events, and predicate results to verify that differences do not arise from different primitive observations. Planning replay supplies identical frontier demands, provider profiles, retained-data availability, deployment state, and operating-condition changes without launching providers. This mode supports exhaustive-oracle and beam-width comparisons on bounded checkpoints drawn from the recorded workloads.

The logical device, edge, and cloud tiers run on the implementation described in Sec. IV. Mininet, Open vSwitch, traffic control, and resource limits impose the evaluated network and compute profiles. Transfers between logical nodes traverse the emulated data path rather than a shared-host shortcut. Exact software, hardware, and provider-profile settings are reported in Appendix E.

c) Baselines.: All policies use the same complex-event definitions, traces, provider implementations, thresholds, retained-data limits, node capacities, and measured provider profiles. Table I summarizes the controlled comparisons.

External CEP and video-analytics systems are not used as primary baselines because reproducing their perception stacks and event grammars would change the observation inputs and planning space. The controlled policies instead share one evaluation harness and isolate the effects of progress-derived work, checkpoint-bounded planning, and joint execution-plan selection.

d) Ground truth and metrics.: A prediction is *binding-correct* when the complex-event type, required entity roles, temporal criterion, and spatial criterion match one unmatched ground-truth occurrence. Delay is measured from the earliest valid completion time. The headline recognition metrics are binding-correct F1, timely recall, and detection delay. The headline resource metrics are provider-active time, GPU-seconds, and transmitted bytes. Planner experiments additionally report feasible-plan rate, planning latency, and cost gap from O1. Spatial experiments report timely handoff recall against downstream activation cost, while retrospective experiments report recovery before the complex-event deadline and retained-data expiration. Detailed diagnostic metrics are reported in the appendix or artifact.

TABLE I
CONTROLLED EVALUATION POLICIES. B3 ISOLATES ADAPTATION TO COMPLEX-EVENT PROGRESS, WHILE B4 ISOLATES THE BENEFIT OF JOINT CHECKPOINT-BOUNDED SEARCH. O1 IS AN OFFLINE ORACLE.

Policy	Key distinction
B0: Always-on	Keeps the full feasible task-level provider/source set active for the replay.
B1: Handwritten static	Uses one developer-authored source, provider, placement, and representation plan; never replans.
B2: Whole-event static	Uses the planner once at admission for the complete request and freezes the resulting plan.
B3: Resource-adaptive	Replans after network or resource changes, but retains the complete task-level work set and ignores hypothesis progress.
B4: Greedy frontier	Uses the same active frontier as *FABLE* but selects each demand independently by immediate execution and transfer cost.
Full *FABLE*	Jointly plans the current frontier through the next semantic checkpoint and replans after semantic or operating changes.
O1: Exhaustive oracle	Enumerates the same checkpoint alternatives on bounded offline cases.

Comparisons are paired on the same trace, replay interval, operating-condition schedule, provider thresholds, retention horizon, and random seed. Results are reported per event family and macro-averaged across families; confidence intervals are computed across independent sessions or replay seeds, not frames from one recording.

B. RQ1: End-to-End Effectiveness

Question. Does progress-driven observation and execution planning preserve complex-event recognition quality and timeliness while reducing computation and communication?

We compare B0, B1, B3, and full *FABLE* on representative multimodal, cross-sensor, alternative, and repeated-visit workloads. End-to-end replay measures binding-correct F1, timely recall, delay, provider-active time, GPU-seconds, and transmitted bytes. The common-observation control verifies that the policies produce equivalent recognition when supplied with the same primitive observations. The primary result is a quality–cost comparison: timely recall and F1 versus provider-active time, GPU-seconds, and communication. We also report work performed for sensors or event branches that were not needed by the realized complex-event occurrence.

C. RQ2: Checkpoint-Bounded Planning

Question. Does jointly selecting sensing sources, inference pipelines, data representations, and execution place-

ments through the next semantic checkpoint improve plan feasibility and cost?

We compare B2, B4, full *FABLE*, and O1 in planning replay and selected end-to-end cases. Controlled ablations fix the provider, placement, representation, or continuation choice. Across checkpoints from the recorded workloads, we vary the eligible sensing sources and provider alternatives, network conditions, provider load, and beam width. We report feasible-plan rate, deadline misses, checkpoint completion time, planning latency, labels generated and pruned, transmitted bytes, retained-artifact reuse, and the plan-cost gap from O1.

a) Planner overhead and sensitivity.: Planner overhead is reported as a function of the number of active hypotheses and frontier demands observed in the recorded workloads, together with the configured number of eligible sensing sources, provider alternatives, and placements. We report p50 and p95 planning latency, labels retained by the beam, planner CPU and memory, and sensitivity to beam width. These experiments measure whether planning remains practical as the execution alternative space grows; they do not claim a separate multi-tenant admission contribution.

D. RQ3: Adaptation to Dynamic Requirements

a) Operating-condition changes.: We compare B2, B3, and full *FABLE* under within-run changes in bandwidth, latency, loss, provider load, and node or provider availability. Each change is injected at a recorded event-time offset. We measure timely recall during and after the change, deadline misses, condition-to-plan latency, provider readiness, and additional communication. B3 isolates resource-aware replanning; comparison with full *FABLE* measures the additional effect of adapting the active work to complex-event progress.

b) Spatial progression.: For eligible 2024 and 2025 fixed-sensor traces, an upstream observation creates a coordination episode when the ongoing match changes which downstream sensors are relevant. We compare broadcast activation, a fixed topology shortlist, resource-only selection, full *FABLE*, and a trace oracle. The main comparison is timely handoff recall versus downstream provider-active time or transmitted bytes. Sensor-targeting precision, false activation, identity handoff accuracy, and provider-ready lead time are supporting metrics.

c) Temporal dependence and continuation.: We compare three variants: no retrospective processing, replay from retained raw segments without reusable intermediate artifacts, and full *FABLE* with typed continuation artifacts and bounded replay. Workloads include cross-camera following, repeated visits, and robbery traces in which a later audio observation makes an earlier vehicle interval relevant. We measure binding-correct recovery before the complex-event deadline and buffer expiration, replay compute, transferred bytes, and identity or role preservation.

Correctness tests for duplicate or stale results, provider failure, and expired retained data are reported separately from the three research questions. Transparent reconstruction of every in-flight hypothesis after controller failure is outside the evaluated guarantee.

VII. Discussion and Limitations

a) Centralized control and persistence.: FABLE uses a logically centralized orchestrator to provide one authoritative ordering of instance transitions, bindings, and demand versions. A standalone MongoDB server stores durable control state and artifact metadata, but does not independently advance CE automata. This design simplifies cancellation, late-result handling, and cross-task sharing, although the orchestrator and database remain potential bottlenecks and failure domains. The evaluation therefore measures scheduler throughput, database update latency, control-plane CPU and memory, and restart recovery. Replicated MongoDB and hierarchical or replicated orchestrators are compatible with the abstraction but remain future work.

b) Messaging and failure detection.: Low-rate control commands and predicate results use MQTT QoS 1, while high-rate telemetry and recurring heartbeats use QoS 0. QoS 1 provides at-least-once delivery rather than exactly-once execution, so stable identifiers, instance-version checks, and idempotent state updates remain necessary. Nodes publish an unconditional one-second heartbeat. The orchestrator marks a node suspect after three consecutive misses and unavailable after five, then replans its active work. These thresholds are appropriate for the current LAN testbed, but deployments with intermittent wireless connectivity may require longer or adaptive timeouts.

c) Provider correctness and profiles.: FABLE does not maintain a probabilistic CE belief state or fuse runtime confidence across providers. Providers expose thresholded predicate matches under measured contracts, so false-positive or false-negative primitive results can create, advance, or miss partial instances. The evaluation therefore varies primitive-provider configurations or injects controlled predicate errors. Scheduling quality also depends on measured cold-start, runtime, serialization, transfer, and resource profiles. Profiles are collected independently from the end-to-end traces and validated against physical execution.

d) Retention and retrospective execution.: Retrospective execution recovers only evidence still present in the configured node-local segment store or artifact repository. The initial configuration retains raw sensor data for approximately 30 minutes and higher-order artifacts for approximately two hours. Longer retention improves recoverability but consumes storage and increases privacy exposure. Raw streams remain local to their originating sensor nodes, while MongoDB stores metadata and references. Once raw data and prerequisite artifacts expire, *FABLE* cannot reconstruct a missed predicate.

e) Coverage and sensing quality.: Negative predicates require operational coverage: the provider must be active, the source stream must progress without unacceptable gaps, and event-time progress must pass the requested interval. The initial runtime does not prove that a functioning camera had an unobstructed view or that an active microphone was semantically informative under interference. Modality-specific health providers, such as low-light, frozen-camera, clipping, or obstruction detectors, can invalidate a coverage interval, but are not required by the core architecture.

f) Bounded state and expressiveness.: The per-task bound K prevents unbounded partial-instance growth, but may remove a candidate that would later become a true event. Group cardinality and membership semantics are compiled from the request; the runtime avoids enumerating every satisfying subset and normally freezes participant groups after matching. The vocabulary targets typed spatiotemporal patterns and does not perform open-ended event discovery or synthesize arbitrary provider chains at runtime.

g) Trust, privacy, and LLM assistance.: *FABLE* enforces declared placement and data-flow constraints when enumerating plans, but does not provide cryptographic confidentiality, remote attestation, or protection from malicious providers. Retained raw streams, descriptors, and state snapshots may themselves be sensitive. GPT-5-mini is considered only as an optional slow-path compiler or advisory component. Its outputs are schema-validated, time-bounded hints and cannot override hard constraints, alter event semantics, or directly advance a partial instance.

VIII. Conclusion

We presented *FABLE*, a runtime that uses the active frontier of each bound partial complex-event instance to orchestrate distributed evidence computation. Unlike task-level adaptive pipelines, *FABLE* creates predicate demands from instance-specific bindings, event-time windows, deadlines, and provider-state requirements. It then selects among pre-registered provider chains and controls their activation, placement, sharing, pre-warming, cancellation, and typed state handoff.

The design treats the complex-event automaton as both a recognizer and a runtime control abstraction. Bounded retrospective execution allows a newly selected provider to reconstruct missing prerequisite evidence while retained data remain available. Versioned instances, stable demand identifiers, leases, event-time progress, and idempotent state updates provide a practical basis for handling late results, duplicate delivery, and node or provider failure without adding probabilistic inference to the orchestration layer.

The completed evaluation will test whether instance-bound frontier scheduling preserves complex-event quality and timeliness while reducing provider activity and data movement relative to static and task-level adaptive execution. It will also quantify control-plane overhead, recovery behavior, profile-based emulation fidelity, and scaling under concurrent complex-event requests.

References

[1] D. Gyllstrom, E. Wu, H.-J. Chae, Y. Diao, P. Stahlberg, and G. Anderson, "SASE: Complex Event Processing over Streams," Dec. 2006, arXiv:cs/0612128. [Online]. Available: http://arxiv.org/abs/cs/0612128

[2] A. Demers, J. Gehrke, B. Panda, M. Riedewald, V. Sharma, and W. White, "Cayuga: A General Purpose Event Monitoring System."

[3] M. Bucchi, A. Grez, A. Quintana, C. Riveros, and S. Vansummeren, "Core: a complex event recognition engine," *arXiv preprint arXiv:2111.04635*, 2021.

[4] M. Ma, E. Bartocci, E. Lifland, J. Stankovic, and L. Feng, "SaSTL: Spatial Aggregation Signal Temporal Logic for Runtime Monitoring in Smart Cities," in *2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPS)*, Apr. 2020, pp. 51–62, iSSN: 2642-9500.

[5] E. Bartocci, L. Bortolussi, M. Loreti, and L. Nenzi, "Monitoring mobile and spatially distributed cyber-physical systems," in *Proceedings of the 15th ACM-IEEE International Conference on Formal Methods and Models for System Design*, 2017, pp. 146–155.

[6] P. Yadav and E. Curry, "Vidcep: Complex event processing framework to detect spatiotemporal patterns in video streams," in *2019 IEEE International conference on big data (big data)*. IEEE, 2019, pp. 2513–2522.

[7] T. Xing, M. R. Vilamala, L. Garcia, F. Cerutti, L. Kaplan, A. Preece, and M. Srivastava, "Deepcep: Deep complex event processing using distributed multimodal information," in *2019 IEEE international conference on smart computing (SMARTCOMP)*. IEEE, 2019, pp. 87–92.

[8] X. Liu, P. Ghosh, O. Ulutan, B. Manjunath, K. Chan, and R. Govindan, "Caesar: cross-camera complex activity recognition," in *Proceedings of the 17th Conference on Embedded Networked Sensor Systems*, 2019, pp. 232–244.

[9] B. Ottenwälder, B. Koldehofe, K. Rothermel, and U. Ramachandran, "Migcep: Operator migration for mobility driven distributed complex event processing," in *Proceedings of the 7th ACM international conference on Distributed event-based systems*, 2013, pp. 183–194.

[10] M. Luthra, B. Koldehofe, P. Weisenburger, G. Salvaneschi, and R. Arif, "Tcep: Adapting to dynamic user environments by enabling transitions between operator placement mechanisms," in *Proceedings of the 12th ACM International Conference on Distributed and Event-based Systems*, 2018, pp. 136–147.

[11] J. Jiang, G. Ananthanarayanan, P. Bodik, S. Sen, and I. Stoica, "Chameleon: scalable adaptation of video analytics," in *Proceedings of the 2018 conference of the ACM special interest group on data communication*, 2018, pp. 253–266.

[12] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "Noscope: optimizing neural network queries over video at scale," *arXiv preprint arXiv:1703.02529*, 2017.

[13] J. Yi, U. G. Acer, F. Kawsar, and C. Min, "Argus: Enabling cross-camera collaboration for video analytics on distributed smart cameras," *IEEE Transactions on Mobile Computing*, vol. 24, no. 1, pp. 117–134, 2024.

[14] P. Yadav, D. Salwala, D. P. Das, and E. Curry, "Knowledge graph driven approach to represent video streams for spatiotemporal event pattern matching in complex event processing," *International Journal of Semantic Computing*, vol. 14, no. 03, pp. 423–455, 2020.

[15] C. Han, C. Chang, S. Srivastava, Y. Lu, and E. Lo, "Scalable complex event processing on video streams," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–29, 2025.

[16] T. Xing, L. Garcia, M. R. Vilamala, F. Cerutti, L. Kaplan, A. Preece, and M. Srivastava, "Neuroplex: learning to detect complex events in sensor networks through knowledge injection," in *Proceedings of the 18th conference on embedded networked sensor systems*, 2020, pp. 489–502.

[17] M. R. Vilamala, T. Xing, H. Taylor, L. Garcia, M. Srivastava, L. Kaplan, A. Preece, A. Kimmig, and F. Cerutti, "Deepprobcep: A neuro-symbolic approach for complex event processing in adversarial settings," *Expert Systems with Applications*, vol. 215, p. 119376, 2023.

[18] A. Amir, I. Kolchinsky, and A. Schuster, "Dlacep: A deep-learning based framework for approximate complex event processing," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 340–354.
[19] L. Han and M. B. Srivastava, "An empirical evaluation of neural and neuro-symbolic approaches to real-time multimodal complex event detection," *arXiv preprint arXiv:2402.11403*, 2024.
[20] L. Han, G. Dong, X. Ouyang, L. Kaplan, F. Cerutti, and M. Srivastava, "Toward foundation models for online complex event detection in cps-iot: A case study," in *Proceedings of the 2nd International Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things*, 2025, pp. 1–6.
[21] O. Ntouni, D. Banelas, and N. Giatrakos, "Neuroflinkcep: Neurosymbolic complex event recognition optimized across iot platforms," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5355–5358, 2025.
[22] M. R. Amer, D. Xie, M. Zhao, S. Todorovic, and S.-C. Zhu, "Cost-sensitive top-down/bottom-up inference for multiscale activity recognition," in *European Conference on Computer Vision*. Springer, 2012, pp. 187–200.
[23] M. R. Amer, S. Todorovic, A. Fern, and S.-C. Zhu, "Monte carlo tree search for scheduling activity recognition," in *Proceedings of the IEEE international conference on computer vision*, 2013, pp. 1353–1360.
[24] Z. Wu, C. Xiong, C.-Y. Ma, R. Socher, and L. S. Davis, "Adaframe: Adaptive frame selection for fast video recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 1278–1287.
[25] R. Vaisenberg, S. Mehrotra, and D. Ramanan, "Exploiting semantics for scheduling data collection from sensors on real-time to maximize event detection," in *Proceedings of SPIE. SPIE*, 2009.
[26] S. Jain, X. Zhang, Y. Zhou, G. Ananthanarayanan, J. Jiang, Y. Shu, P. Bahl, and J. Gonzalez, "Spatula: Efficient cross-camera video analytics on large camera networks," in *2020 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 2020, pp. 110–124.
[27] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi, "Naiad: a timely dataflow system," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 2013, pp. 439–455.
[28] T. Akidau, A. Balikov, K. Bekiroğlu, S. Chernyak, J. Haberman, R. Lax, S. McVeety, D. Mills, P. Nordstrom, and S. Whittle, "Millwheel: fault-tolerant stream processing at internet scale," *Proc. VLDB Endow.*, vol. 6, no. 11, p. 1033–1044, Aug. 2013. [Online]. Available: https://doi.org/10.14778/2536222.2536229
[29] R. Avnur and J. M. Hellerstein, "Eddies: Continuously adaptive query processing," in *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, 2000, pp. 261–272.
[30] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE transactions on parallel and distributed systems*, vol. 13, no. 3, pp. 260–274, 2002.
[31] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proceedings of the ACM special interest group on data communication*, 2019, pp. 270–288.
[32] M. Hoffmann, A. Lattuada, F. McSherry, V. Kalavri, J. Liagouris, and T. Roscoe, "Megaphone: Latency-conscious state migration for distributed streaming dataflows," *Proceedings of the VLDB Endowment*, vol. 12, no. 9, pp. 1002–1015, 2019.
[33] B. Del Monte, S. Zeuch, T. Rabl, and V. Markl, "Rhino: Efficient management of very large distributed state for stream processing engines," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 2471–2486.
[34] F. McSherry, A. Lattuada, M. Schwarzkopf, and T. Roscoe, "Shared arrangements: practical inter-query sharing for streaming dataflows," *Proc. VLDB Endow.*, vol. 13, no. 10, p. 1793–1806, Jun. 2020. [Online]. Available: https://doi.org/10.14778/3401960.3401974
[35] H. Zhang, G. Ananthanarayanan, P. Bodik, M. Philipose, P. Bahl, and M. J. Freedman, "Live video analytics at scale with approximation and {Delay-Tolerance}," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 377–392.
[36] D. Kang, P. Bailis, and M. Zaharia, "Blazeit: optimizing declarative aggregation and limit queries for neural network-based video analytics," *arXiv preprint arXiv:1805.01046*, 2018.
[37] C. Canel, T. Kim, G. Zhou, C. Li, H. Lim, D. G. Andersen, M. Kaminsky, and S. Dulloor, "Scaling video analytics on constrained edge nodes," *Proceedings of Machine Learning and Systems*, vol. 1, pp. 406–417, 2019.
[38] Y. Li, A. Padmanabhan, P. Zhao, Y. Wang, G. H. Xu, and R. Netravali, "Reducto: On-camera filtering for resource-efficient real-time video analytics," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 359–376.
[39] G. Gao, Y. Dong, R. Wang, and X. Zhou, "Edgevision: Towards collaborative video analytics on distributed edges for performance maximization," *IEEE Transactions on Multimedia*, vol. 26, pp. 9083–9094, 2024.
[40] H. Shen, L. Chen, Y. Jin, L. Zhao, B. Kong, M. Philipose, A. Krishnamurthy, and R. Sundaram, "Nexus: A gpu cluster engine for accelerating dnn-based video analysis," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 322–337.
[41] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "{INFaaS}: Automated model-less inference serving," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 397–411.
[42] X. Fu, T. Ghaffar, J. C. Davis, and D. Lee, "{EdgeWise}: A better stream processing engine for the edge," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 929–946.
[43] A. H. Jiang, D. L.-K. Wong, C. Canel, L. Tang, I. Misra, M. Kaminsky, M. A. Kozuch, P. Pillai, D. G. Andersen, and G. R. Ganger, "Mainstream: Dynamic {Stem-Sharing} for {Multi-Tenant} video processing," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018, pp. 29–42.
[44] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P. Aditya, and V. Hilt, "{SAND}: towards {High-Performance} serverless computing," in *2018 USENIX annual technical conference (USENIX ATC 18)*, 2018, pp. 923–935.
[45] V. Sreekanti, C. Wu, X. C. Lin, J. Schleier-Smith, J. E. Gonzalez, J. M. Hellerstein, and A. Tumanov, "Cloudburst: stateful functions-as-a-service," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 2438–2452, Jul. 2020. [Online]. Available: https://doi.org/10.14778/3407790.3407836
[46] E. Zhang, N. Sullivan, B. Haynes, R. Krishna, and M. Balazinska, "Self-enhancing video data management system for compositional events with large language models," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–29, 2025.

APPENDIX

A. Request Resolution and Semantic Representation

A request selects an authored complex-event family and a validated set of parameters. The current prototype includes families for convoy, robbery, rendezvous or package exchange, drive-up shooting, repeated visit or stalking, vehicle convergence, and two-vehicle chase. An authored alias selects one of those families; structured parameters choose permitted variants, lookback intervals, visit counts, or return windows. Request resolution does not invent a new event definition during execution. It invokes an authored graph builder, assigns a graph version, and initializes the shared temporal event graph used by the semantic engine.

The request-level representation is a typed AND/OR temporal graph. Table II summarizes the main constructs. Primitive predicates describe semantic conditions rather than software operators. Composite nodes and temporal guards allow several event branches to share common prefixes and suffixes while retaining explicit ordering and timing semantics.

B. Hypothesis Lifecycle and Event Time

A matching seed observation creates a hypothesis. Subsequent predicate results can update, fork, merge, invalidate, expire, or complete the hypothesis. The semantic

Construct	Meaning
Primitive predicate	A typed semantic condition such as `PASSES`, `FOLLOWS`, `TRANSFER`, or `AUDIO_EVENT`.
AND / K-of-N	Concurrent or unordered conditions that must all, or a specified number, be satisfied.
OR	Authored alternative complex-event branches that can share common prefixes and suffixes.
Sequence edge	Enables a condition after a semantic predecessor is satisfied.
Temporal guard	Bounds precedence, overlap, duration, absence, repetition, or maximum gap.
Role constraint	Defines entity types, distinctness, binding introduction, validation, and fork behavior.

TABLE II
PRINCIPAL SEMANTIC GRAPH CONSTRUCTS.

Demand field	Purpose
Attribution	Request, graph, hypothesis, frontier, and checkpoint versions.
Predicate and bindings	Required semantic result and the bound or introducible entity roles.
Event-time interval	Past, live, or future interval whose observations must be examined.
Latest useful completion	Wall-clock deadline after which the result cannot affect the hypothesis.
Source scope	Eligible sources and optional preferences derived from deployment or route knowledge.
Capabilities and outputs	Required semantic capability and acceptable typed result or artifact formats.
Hard constraints	Data locality, transfer, placement, retention, quality-floor, and continuation requirements.

TABLE III
PRINCIPAL FIELDS OF A PREDICATE DEMAND.

runtime stores canonical entity bindings, per-node status, viable branches, temporal anchors and windows, observation provenance, and references to retained artifacts. Binding-introducing results may create bounded child hypotheses, while canonically equivalent hypotheses are merged. The current implementation uses a configurable per-frontier fork limit rather than a global limit on the number of hypotheses for one request.

Temporal guards use event time. Duration predicates require a condition to hold through an interval. Absence succeeds only after the relevant sources provide operational coverage and source progress passes the end of the interval plus allowed lateness. A failed child does not invalidate a hypothesis when another authored branch remains viable. Returned results are accepted only when their request, graph version, hypothesis version, bindings, frontier or checkpoint, and event-time interval still match the current semantic state.

A semantic checkpoint is the nearest primitive or composite result that may change bindings, viable branches, cancellation decisions, future obligations, or required intermediate artifacts. Examples include a primitive match, completion of an AND node, resolution of an OR branch, a cardinality boundary, or closure of an absence window. Planning only through the next checkpoint avoids committing to the rest of the complex-event workflow before the observations needed to choose that workflow are available.

C. Predicate Demands and Provider Contracts

The Active Frontier Manager converts each enabled primitive predicate into a provider-independent predicate demand. Table III lists the principal fields. The event-time interval and latest useful completion time are intentionally separate: the first specifies when the physical observations occurred or will occur, while the second specifies when computation must finish for the result to affect recognition.

A provider contract describes a detector, tracker, relation evaluator, or short inference pipeline that can satisfy one or more semantic predicates. Contracts declare typed inputs and outputs, role-binding behavior, eligible node classes, resource requirements, replayability, model and feature compatibility, quality floors, and any intermediate artifacts that can be consumed or produced. Raw sensor media, detections, tracks, selected crops, identity descriptors, and audio features are represented as distinct data types rather than as interchangeable scalar-quality levels.

For one predicate demand, a candidate physical alternative combines a sensing source or retained input, a compatible provider chain, a data representation, an execution placement, required transfers, and intermediate artifacts. These choices are coupled. Same-camera following can operate on local tracks, whereas cross-camera association may require an identity descriptor to be produced and transferred before the downstream relation can be evaluated.

D. Checkpoint-Bounded Multi-Label Planning

The execution planner jointly chooses alternatives for the demands needed through one semantic checkpoint. A search label is a partial execution plan, not a complex-event hypothesis. The implementation performs the following steps at each demand boundary:

1) extend every surviving label with each compatible physical alternative for the next demand;
2) reject labels that violate provider schemas, bindings, data locality, network reachability, resource capacity, retention, quality floors, continuation requirements, or latest useful completion;
3) remove deterministic duplicates and labels dominated at the same demand boundary; and
4) apply a stable ranking and retain at most beam width K labels for further expansion.

Two labels are compared for dominance only when they cover the same demand prefix and preserve equivalent checkpoint semantics and bindings. One label dominates another when the first is no worse in predicted completion, deadline slack, resource use, transferred bytes,

source preference, minimum quality, and continuation coverage, and is strictly better in at least one dimension. A latency–quality tradeoff can therefore leave both labels non-dominated when both satisfy the required quality floor.

Hard feasibility is enforced before ranking. The prototype then applies a stable lexicographic order over source-preference penalty, predicted checkpoint completion, deadline slack, observation perishability, startup and resource cost, transmitted bytes, missing desired continuation types, and a stable identifier. Beam width bounds planning time; a bounded exhaustive mode supports small offline oracle comparisons and logs generated, rejected, dominated, and beam-pruned labels.

E. Runtime Control, Reuse, and Cancellation

The planner returns a primary checkpoint plan and bounded fallbacks. Runtime control verifies current capacity, materializes the selected provider steps, and attaches demand-scoped leases to provider instances and artifacts. Reuse is permitted only when the provider configuration, input source or artifact, event-time interval, output schema, representation version, and placement are compatible. Separate hypotheses can consume the same low-level result while retaining different bindings and downstream state.

A lease is released when the corresponding predicate demand is satisfied, expires, or is removed from the active frontier. Releasing one lease does not stop a provider that still has another valid consumer. The controller detaches or stops work when an event branch becomes impossible, a hypothesis terminates, or a result can no longer arrive in time to affect recognition. Provider or node failure invalidates the current execution plan rather than the semantic event graph; a changed resource or network epoch can cause the planner to use a fallback or construct a new checkpoint plan.

Retrospective work is bounded by the demand interval, retained-data coverage, provider replayability, a configured lookback limit, and the latest useful completion time. Results preserve the original event timestamps of the observations being analyzed. Spatial preferences are similarly hypothesis-specific: route and deployment knowledge may rank downstream sensing locations, but an unavailable or uncertain topology produces neutral source preferences or a bounded set of alternatives rather than a semantic failure.

F. Software Organization and Configuration

The prototype is implemented in Python using validated typed records for identifiers, intervals, graph state, predicate demands, provider contracts, artifacts, plans, commands, and results. Authored event families are Python graph builders. Provider, chain, data-type, deployment, and runtime definitions are loaded from versioned YAML files. The implementation separates semantic state, physical planning, runtime control, and distributed execution, although the main paper presents those modules through the semantic-engine, execution-planner, and node-agent components shown in Fig. 2.

The demand compiler combines predicate schemas with current bindings, event-time windows, source restrictions, data-locality rules, transfer limits, and continuation requirements. When a site-specific qualitative transition model is available, current source, heading, corridor, and route information produce ranked source preferences. Those preferences are soft by default, so an uncertain route does not eliminate otherwise feasible sensors.

The default online configuration uses beam width eight and retains two fallback plans. Retrospective work is bounded to a 30-minute lookback unless an experiment manifest specifies another limit. A bounded exhaustive planner is available for small offline comparisons and is capped at 50,000 combinations by default. These values are configuration choices rather than event semantics.

G. Messaging, State, and Fault Handling

Node agents execute managed Docker providers, adopt configured replay or analytics services, or run deterministic reference providers. Commands use allowlisted images, commands, mounts, devices, environment variables, and resource limits; agents do not execute arbitrary controller-supplied shell commands. The controller and agents communicate through versioned MQTT messages for provider activation, predicate results, status updates, and artifact announcements. A SQLite outbox retries unacknowledged messages, and a processed-message ledger provides idempotent handling above MQTT QoS.

Control state has in-memory and MongoDB-backed interfaces, while local retained segments are indexed in SQLite by source and event-time coverage. Heartbeats report node and provider availability and can advance a resource epoch that triggers replanning. Session and sequence provenance prevent stale heartbeats or results from a previous run from being accepted. Provider and lease reconciliation are implemented; transparent reconstruction of every in-flight hypothesis after controller failure is outside the evaluated guarantee.

H. Optional Language-Model Interfaces

A language model is not required in the fast path. An optional request interpreter may map free-form language to an authored family and validated parameters, but cannot emit an executable graph, provider plan, code, or shell command. An optional checkpoint advisor may return a bounded, expiring hint that ranks currently valid event branches or explains a planning failure. Schema validation, graph transitions, temporal closure, provider compatibility, placement feasibility, and final complex-event acceptance remain deterministic. A separately configured visual-language-model provider can be used as a bounded identity fallback; it returns an ordinary typed

predicate result and has no authority over event semantics.

I. Implemented Predicates and Provider Realization

FABLE exposes semantic predicates to the event specification while hiding the provider-specific detections, tracks, embeddings, and intermediate state used to produce them. Table IV therefore includes only request-level predicates with an implemented provider path. Single-frame object detections are also implemented and appear later as atomic evidence, but they are omitted from this table because they are intermediate observations rather than authored semantic predicates. The provider column summarizes the major predicate-specific stages rather than catalog identifiers or service names.

We use two coarse temporal classes based on the evidence required to establish a predicate. *Instant* denotes a predicate evaluated for one event time, video frame, or short audio window. *Over time* denotes a predicate that requires multiple observations, a sustained interval, or a bounded historical search. This classification concerns provider evidence, not whether the event graph ultimately receives a state or transition result. Likewise, the sensor column distinguishes predicates that use one sensing context from those that combine multiple physical sensors. All core provider paths listed in the table are deployable as Docker services; optional hosted fallbacks are omitted.

J. Predicate Grammar and Spatiotemporal Classification

A complex-event request is written in terms of implemented semantic predicates, rather than provider names. Each predicate binds entities and, when needed, a zone, line, sensor, route, time window, or threshold. The temporal AND/OR event graph combines predicate results through conjunction and alternatives, ordering and bounded windows, persistence and absence, and quantification over bound entities or conditions.

Figure 4 organizes request-level predicates together with selected internal evidence according to two reader-facing dimensions: whether the evidence is obtained at an instant or over time, and whether it comes from one sensor or scene or from multiple sensors or scenes. The figure therefore includes YOLO object detections as atomic observations even though they are not normally exposed as authored predicates.

An atomic observation may be used directly as a leaf in a complex-event graph, or it may provide evidence for a temporal or cross-sensor predicate. For example, a single `AUDIO_EVENT` may directly satisfy one branch of a robbery pattern, while repeated object observations may be tracked to establish `MOVING`, `ENTERS`, or `FOLLOWS`. The intermediate categories are therefore available compositions, not mandatory processing stages.

The classification is based on the evidence needed to establish a result. Consequently, `INSIDE` is atomic because zone membership can be evaluated at one track

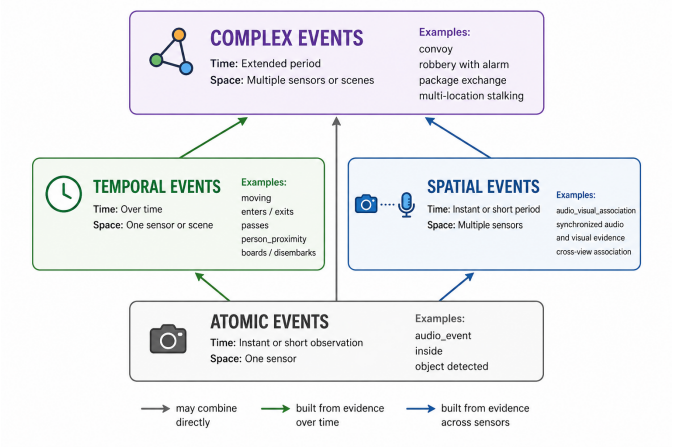

Fig. 4. Spatiotemporal organization of predicate evidence. Atomic observations may be used directly in a complex-event graph or combined into predicates that aggregate evidence over time or across sensors. Green and blue arrows denote temporal and cross-sensor construction, respectively; the gray arrow denotes direct use in a complex-event pattern.

snapshot. The state may persist, but persistence is imposed separately through repeated evaluation or a duration operator. By contrast, `ENTERS` and `EXITS` require a change across observations and are therefore temporal, even though the resulting transition is reported at one event time.

Table V maps the implemented predicates and selected internal evidence to the four categories shown in Figure 4. The categories summarize typical evidence scope rather than defining a strict software call graph.

TABLE IV
IMPLEMENTED PREDICATE–PROVIDER RELATIONSHIPS. PROVIDER COUNTS DESCRIBE MAJOR PREDICATE-SPECIFIC STAGES AFTER STANDARD SENSOR INGESTION; SHARED TRANSPORT, SERIALIZATION, AND ORCHESTRATION ARE NOT COUNTED.

Predicate(s)	Meaning	Time	Sensors	Major provider stages	Execution
AUDIO_EVENT	Detects a named sound event.	Instant	1	1 stage: YAMNet audio classification, with a lightweight spectral alternative.	ML or rules; Dockerized
AUDIO_VISUAL_ASSOCIATION	Associates a sound with visible activity.	Instant	Multiple	3 stages: audio classification, GCC-PHAT localization, and audio–visual matching over existing visual tracks.	ML + signal processing + rules; Dockerized
INSIDE	Tests whether an entity lies within a zone.	Instant	1	1 stage over tracks: polygon or zone-membership evaluation.	Rules; Dockerized
MOVING	Determines whether an entity is moving.	Over time	1	1 stage over track history: displacement or speed thresholding.	Rules; Dockerized
DISTANCE_LT	Tests whether two entities remain closer than a threshold.	Over time	1	1 stage over aligned tracks: metric distance or an image-space approximation.	Rules; Dockerized
ENTERS, EXITS	Detects movement into or out of a zone.	Over time	1	2 stages: zone membership followed by transition detection.	Rules; Dockerized
PASSES	Detects passage across a line or reference.	Over time	1	1 stage over a track path: directed line-crossing or traversal evaluation.	Rules; Dockerized
PERSON_PRESENT, PERSON_PROXIMITY	Detects sustained person presence or proximity.	Over time	1	1 relation stage over detected and tracked people.	ML + rules; Dockerized
BOARDS, DISEMBARKS, DEPARTURE_OR_ESCAPE	Detects person–vehicle transitions and departure.	Over time	1	1 state-machine stage over detected and tracked people and vehicles.	ML + rules; Dockerized
FOLLOWS	Detects sustained or sequential following.	Over time	1 or multiple	Local: 1 geometric reasoning stage. Multi-sensor: 3 main stages for descriptor extraction, identity association, and following reasoning.	Rules, with ML identity matching when needed; Dockerized
CONVERSATION, THREAT_EVENT	Detects a conversation or threat-related interaction.	Over time	Multiple	5 core stages: visual proximity, voice activity, speaker description, speaker-turn analysis, and interaction reasoning; optional speech-content recognition adds one stage.	ML + rules; Dockerized
TRANSFER	Detects transfer of a package or object between entities.	Over time	1	3 stages: package detection, interaction scoring, and custody-change reasoning over tracked entities.	ML + rules; Dockerized
PERSON_ENTERED_BEFORE, PERSON_PRESENT_BEFORE, VEHICLE_PRESENT_BEFORE	Recovers relevant presence or entry before a later trigger.	Over time	1 or multiple	1–3 main stages: historical interval matching, with retained-data detection/tracking and cross-sensor identity association added when needed.	ML + rules; Dockerized

TABLE V
SPATIOTEMPORAL CLASSIFICATION OF IMPLEMENTED SEMANTIC PREDICATES AND SELECTED INTERNAL EVIDENCE. CLASSIFICATION IS BASED ON THE OBSERVATIONS REQUIRED TO ESTABLISH A RESULT, NOT ON WHETHER THE RESULT IS EMITTED AS A STATE OR TRANSITION.

Category	Time	Sensor scope	Implemented predicates or evidence
Atomic observations	Instant or short observation	One sensor	AUDIO_EVENT, INSIDE, and YOLO object detections (OBJECT_PRESENT, internal evidence).
Temporal predicates	Over time	One sensor or scene	MOVING, DISTANCE_LT, ENTERS, EXITS, PASSES, PERSON_PRESENT, PERSON_PROXIMITY, BOARDS, DISEMBARKS, DEPARTURE_OR_ESCAPE, and TRANSFER. A local realization of FOLLOWS also belongs to this category.
Spatial associations	Instant or short period	Multiple sensors	AUDIO_VISUAL_ASSOCIATION. Cross-view identity association (SAME_ENTITY) is also implemented as internal evidence for multi-sensor predicates.
Complex-event predicates and patterns	Extended period	Multiple sensors or scenes	The cross-sensor realization of FOLLOWS; CONVERSATION; THREAT_EVENT; PERSON_ENTERED_BEFORE; PERSON_PRESENT_BEFORE; and VEHICLE_PRESENT_BEFORE. Complete event-graph patterns include convoy, robbery with alarm, package exchange, and multi-location stalking.

Notes: The categories are not disjoint implementation layers. A predicate is placed according to its typical realization. For example, FOLLOWS is temporal when evaluated within one scene and participates in the complex category when it links observations across cameras. Retrospective predicates may search one sensor, but their principal *FABLE* realization can fan out across selected sensors.

TABLE VI
SUMMARY OF THE *FABLE* EVALUATION CORPUS AFTER QUALITY FILTERING. RUNS ARE EXPERIMENTS MARKED AS RECOMMENDED FOR USE. LABELED TIME IS THE GROUND-TRUTH COMPLEX-EVENT WINDOW USED DURING REPLAY, RATHER THAN THE DURATION OF THE ENCLOSING RAW RECORDING.

Year	Complex event	Runs	Total time	Min./med./ max.	Sensor scope	Primary evidence and event structure
2024	Vehicle convergence	5	0:09:35	72/120/166 s	Multiple locations	Multiple vehicles approach a common area across sensor views.
2024	Route convoy	23	0:37:44	81/95/146 s	Multiple locations	Ordered vehicle movement and following across sequential camera views.
2024	Two-vehicle chase	19	0:14:26	38/44/63 s	Multiple locations	One vehicle follows another across cameras using track order and identity.
2025	Robbery with alarm	15	0:20:37	21/73/170 s	Multiple locations	An audio trigger initiates retrospective visual recovery and vehicle identity matching across sensor views.
2025	Vehicle rendezvous	3	0:06:00	60/120/180 s	One location	Two vehicles converge and remain visually close within the same scene.
2025	Talking/rendezvous	5	0:15:14	120/197/258 s	One location	Person proximity is combined with speech activity or speaker-turn evidence from the same scene.
2026	Pass-follow-clear convoy	6	0:11:00	60/120/180 s	Multiple locations	Vehicles pass a reference, form a following pattern, and then leave the monitored area.
2026	Three-visit stalking	3	0:18:00	240/360/480 s	Multiple locations	Repeated visits are linked across time and cameras using appearance and identity evidence.
2026	Cross-sensor robbery	4	0:11:00	120/180/180 s	Multiple locations	An audio trigger is combined with retrospective vehicle recovery across selected cameras.
All recommended experiments		**83**	**2:23:36**	–	–	Nine complex-event types collected across three campaigns.

Notes: Total time is reported as h:mm:ss; per-run durations are minimum/median/maximum in seconds. Sensor scope describes whether the event can be established within one sensing location or requires evidence across multiple locations. Sensor availability indicates that matching media exists, not that every sensor observes the event throughout the complete labeled window.

TABLE VII
SUMMARY OF THE SENSING DEPLOYMENTS USED IN EACH COLLECTION CAMPAIGN.

Year	Deployment	Recorded modalities	Sensor locations
2024	13 sensors: 10 fixed Orin/ZED nodes and 3 mobile nodes.	Fixed stereo RGB/depth video and multichannel audio; mobile synchronized video and mono audio.	Known and fixed per run.
2025	13 sensors: 10 fixed Orin/ZED nodes and 3 mobile nodes.	Fixed stereo RGB/depth video and multichannel audio; mobile synchronized video and mono audio.	Known and fixed per run.
2026	6 fixed sensors from Orin 11–16.	Stereo RGB/depth video, multichannel audio, and GPS where available.	Not reliably known.